

Burrete: Local-first molecular file inspection for macOS, Quick Look, and agent workflows

Sergei A. Nikolenko^{a,b,*}

^a*Ligand Pro LLC, Bolshoy Boulevard 42, Building 1, Skolkovo Innovation Center, Moscow 121205, Russia*

^b*Innopolis University, 1 Universitetskaya Street, Innopolis 420500, Russia*

Abstract

Burrete is an open-source, local-first workspace for inspecting molecular artifacts on macOS. It combines Finder Quick Look, a Tauri desktop workspace, a source-built iPhone preview target, and typed agent interfaces. Rather than introducing a molecular graphics or modeling algorithm, Burrete routes local documents to established components: Mol* for interactive structures and MolViewSpec scenes, RDKit for molecule collections, Ketcher for sketching, optional `xyzrender` for SVG output, VESTA handoff, and bounded text/metadata fallbacks. This integration supports rapid checks of structures, pose collections, trajectories, quantum-chemistry inputs, and workflow outputs before heavier analysis. Version 1.0.22 is released under the MIT License with source, documentation, sample files, and macOS binaries on GitHub.

Keywords: molecular visualization, research software, computational chemistry, cheminformatics, Quick Look, agent workflows

Required Metadata

Current code version

Main text

1. Motivation and significance

Computational chemistry, structural biology, docking, and cheminformatics workflows create heterogeneous local artifacts: coordinate files,

*Corresponding author

Email address: `nikolenko.sergei.chem@gmail.com` (Sergei A. Nikolenko)

URL: `https://orcid.org/0000-0003-1150-9390` (Sergei A. Nikolenko)

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v1.0.22
C2	Permanent GitHub link to code/repository used for this code version	https://github.com/SergeiNikolenko/Burrete/tree/softwarex-v1.0.22
C3	Legal Code License	MIT License
C4	Code versioning system used	Git
C5	Software code languages, tools, and services used	TypeScript, JavaScript, Rust, Swift, HTML/CSS, shell; Tauri 2; Mol* 5.7.0; RDKit 2025.03.4; Ketcher 3.12/3.15
C6	Compilation requirements, operating environments & dependencies	macOS 12 or later; Bun and Rust/Tauri for the desktop source build; Xcode for the Quick Look extension and source-built iPhone target. Mol*, RDKit, and Ketcher assets are bundled. xyzrender and VESTA are optional external tools.
C7	If available Link to developer documentation/manual	https://github.com/SergeiNikolenko/Burrete/blob/softwarex-v1.0.22/README.md
C8	Support email for questions	nikolenko.sergei.chem@gmail.com

Table 1: Code metadata (mandatory).

ligand collections, docking poses, trajectories, quantum-chemistry inputs and outputs, topologies, logs, and generated reports. Before a researcher can interpret a result, a smaller question often has to be answered: does this file contain the expected structure, set of molecules, frames, or workflow state? Repeatedly opening a full modeling application for that check interrupts the file-oriented workflow, while a generic operating-system preview usually lacks molecular semantics.

Established applications and libraries already solve the difficult scientific and graphics problems. ChimeraX and Coot support structural analysis, model-building, and refinement [1, 2]; VMD and PyMOL are broad molecular-visualization environments [3, 4]; Avogadro

supports molecular editing and computational-chemistry workflows [5]; and Open Babel and RDKit provide conversion and cheminformatics capabilities [6, 7]. Web components such as Mol*, NGL, Jmol/JSmol, and 3Dmol.js make molecular visualization embeddable in applications [8, 9, 10, 11].

Burrete addresses a narrower integration problem: it gives the same local artifact an operating-system preview, a persistent desktop workspace, an explicit renderer route, and—when requested—a typed agent session. A typical user selects a molecular file in Finder and presses Space for a Quick Look preview, or opens a project folder in the desktop application to compare files in tabs and docked panels. The contribution is therefore not a new molecular graphics, cheminformatics, docking, or simulation algorithm. It is a documented inspection and handoff layer that reduces friction between file generation and scientific analysis.

2. Software description

2.1 *Software architecture.*

Figure 1 summarizes the architecture. A checked-in format registry defines document types and routing policy. Three user-facing applications consume that policy: a Swift Quick Look extension following Apple’s extension model [12], a React desktop workspace hosted by Tauri [13], and an experimental iPhone preview target built from source. The desktop and Quick Look surfaces package their own runtime artifacts and caches; they are tested separately because a successful browser preview does not establish that macOS has registered the native extension.

The preview adapters make component ownership explicit. Mol* 5.7.0 renders supported coordinate files and MolViewSpec scenes [8, 14]; the packaged RDKit 2025.03.4 WebAssembly runtime parses and depicts molecule collections [7]; and Ketcher provides small-molecule and reaction editing [15]. Optional **xyzrender** produces SVG-oriented molecular graphics [16], while VESTA can be invoked for compatible crystal, volumetric, and quantum-chemistry artifacts [17]. Files without enough standalone structural information remain available through text or metadata views. OpenMM is an example of an upstream simulation ecosystem whose coordinate and state artifacts can be inspected; Burrete does not execute the simulation [18].

The agent platform is a separate typed boundary over the workspace. Repository commands expose **open**, **observe**, **act**, and **render-panel**. Observations report bounded state such as

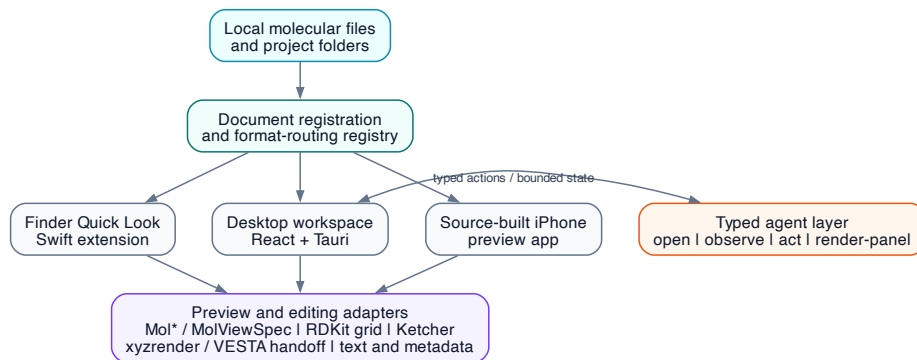


Figure 1: Burrete architecture. A shared document and format registry routes local files to native or desktop surfaces and then to explicit preview or editing adapters. The agent layer exchanges typed actions and bounded state with a workspace session.

active and open documents, viewer readiness, panels, selection, and the last action result. Actions are explicit discriminated operations rather than unrestricted screen control or arbitrary file injection.

2.2 Software functionalities.

The main functions are:

- native registration and Finder Quick Look for supported local molecular documents;
- a tabbed desktop workspace with project folders, recent files, command search, renderer switching, and text fallbacks;
- Mol* previews for common structure formats and declarative MVSJ/MVSX scenes;
- RDKit-backed grids for SDF, SD, SMILES, CSV, and TSV collections with search, sorting, SMARTS filtering/highlighting, selection, append/merge, and export;
- Ketcher small-molecule and reaction sketching, with handoff to preview and collection workflows;
- optional **xyzrender** and VESTA routing for formats and tasks better handled by those external applications;
- inspection of trajectory, topology, FEP-network, quantum-chemistry, OpenMM, Amber, and CHARMM artifacts when sufficient standalone or paired context exists, otherwise a text or metadata view; and

- typed browser or desktop sessions for bounded agent-assisted inspection.

These functions intentionally preserve boundaries. Burrete does not claim automatic chemical interpretation, property prediction, scoring, structure refinement, trajectory analysis, or complete direct rendering of every registered extension.

3. Illustrative examples

The first example is fast structure triage. Selecting a PDB, mmCIF, SDF, or XYZ document in Finder invokes the registered Quick Look path. Opening the same document in the desktop workspace adds persistent tabs, project navigation, renderer controls, selection, and comparison panels. Figure 2 shows a PDB structure in the Mol*-based workspace. The user can verify the macromolecular assembly and ligand context, then keep that view mounted while inspecting related project files.

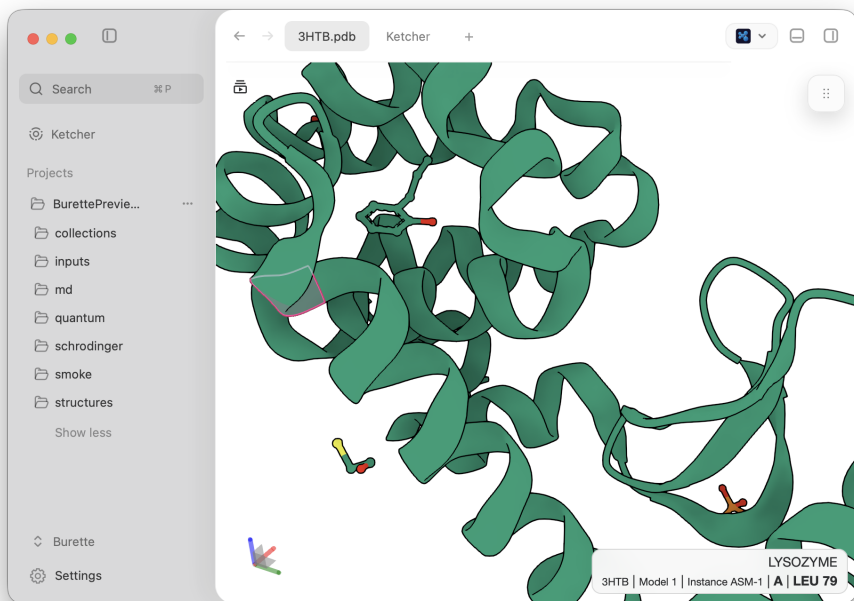


Figure 2: Desktop workspace displaying PDB entry 3HTB with the Mol*-based viewer, project navigation, tabbed documents, and atom-level selection.

The second example is collection triage. An SDF, SMILES, CSV, or TSV file opens as a table of molecule records. Depictions and source fields remain together; the user can search or sort, apply a SMARTS

filter, select records, edit a row in Ketcher, preview it in Mol*, or export a subset. Figure 3 shows the context actions for a selected SDF record. This path is useful for checking generated libraries and docking-input collections before launching downstream computations.

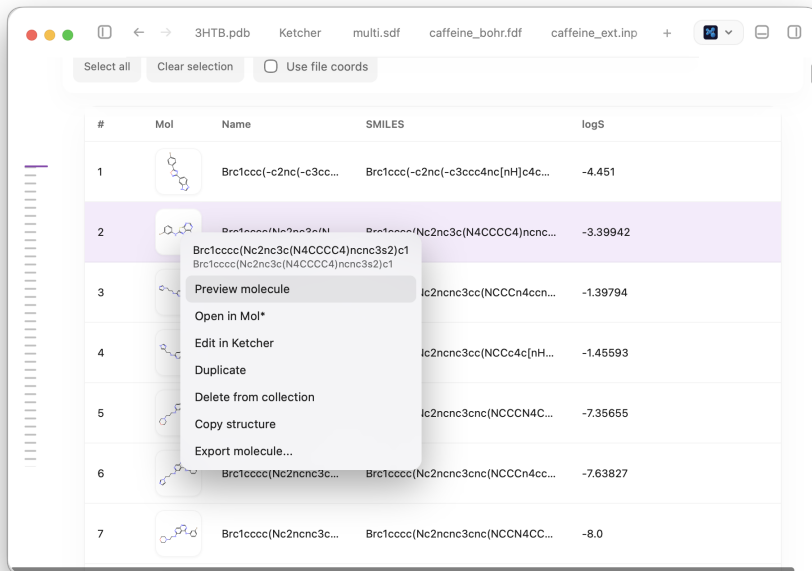


Figure 3: RDKit-backed SDF collection grid. Molecular depictions, source properties, selection, preview, editing, and export remain in one local workspace.

The third example is explicit external rendering. When **xyzrender** is installed, a compatible XYZ or quantum-chemistry artifact can be routed to it without losing the surrounding project context. Figure 4 shows the SVG preview and renderer presets in a docked panel. The same principle applies to VESTA handoff: Burrete owns file discovery and routing, while the specialized external program owns the scientific visualization.

Finally, an automated workflow can open one of these artifacts, inspect the typed session state, request a defined viewer action, and place a bounded markdown, table, or chart result in a docked panel. The machine-readable state remains the primary verification channel; screenshots are reserved for visual quality assurance.

4. Impact

Burrete’s immediate impact is operational: it shortens the transition from a generated file to an informed decision about the next tool. The

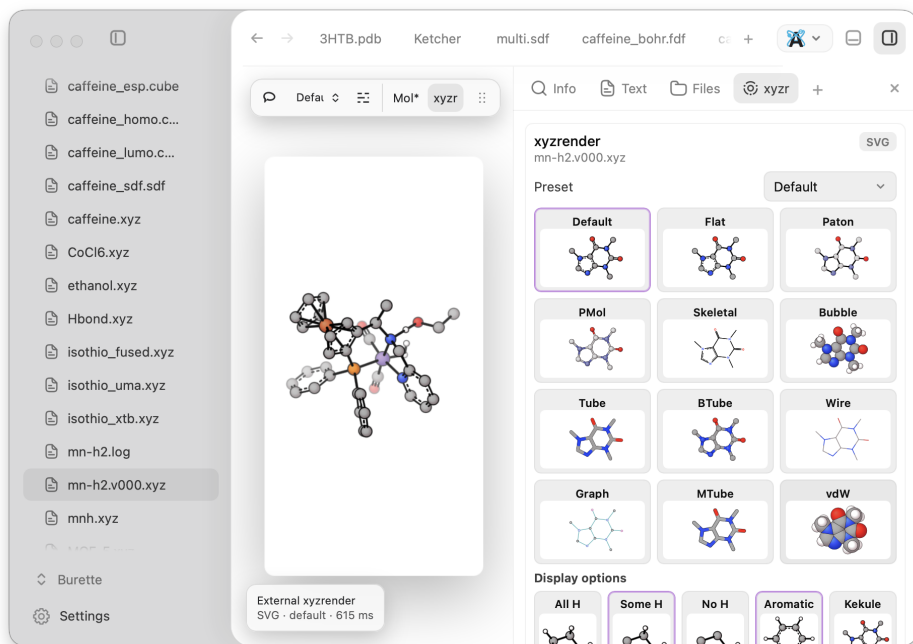


Figure 4: Optional **xyzrender** adapter showing an SVG molecular preview and renderer presets alongside the local project file list.

same workspace can expose a structure, a ligand table, a text log, and a specialized-renderer handoff without implying that these artifacts share one scientific parser. This is particularly useful in research pipelines that produce many intermediate files and where provenance, renderer choice, and failed fallbacks must remain visible.

The typed session boundary also creates a reproducible point of contact for AI-assisted research software. An agent can query explicit document and viewer state and invoke a limited action contract instead of inferring all state from pixels. This enables research questions around auditable human-agent inspection, cross-surface consistency, and bounded context for molecular workflows without positioning the agent as an autonomous scientific decision-maker.

The project is at an early adoption stage. On 10 July 2026, the public repository reported 17 GitHub stars and 148 downloads of binary release assets across releases; these are download events rather than unique-user counts [19]. The author is not aware of a peer-reviewed study that has yet used Burrete as a research method, so no publication-impact claim is made. Instead, the repository supplies source, published macOS release artifacts, sample files, documentation, and con-

tract tests so that prospective users and reviewers can inspect the implementation and reproduce the illustrated workflows.

5. Conclusions

Burrete combines Finder Quick Look, a persistent desktop workspace, a source-built iPhone target, explicit molecular renderer adapters, and typed agent sessions around local molecular files. Its contribution is the integration boundary: the software makes established viewers and editors available in a consistent inspection workflow while preserving which component owns parsing, depiction, editing, or external visualization.

The current release is macOS-first; the iPhone target is experimental and requires a source build. Optional external programs must be installed separately, binary trajectories need appropriate paired context, and some registered workflow artifacts deliberately fall back to text or metadata. Future work will expand tested format coverage, collect public research-use cases, and evaluate inspection workflows with independent users. These limitations are preferable to overstating direct rendering or scientific analysis that the software does not perform.

Acknowledgements

The author thanks the open-source communities behind Mol*, RDKit, Ketcher, Tauri, `xyzrender`, and the other software cited in this article.

CRedit authorship contribution statement

Sergei A. Nikolenko: Conceptualization, Methodology, Software, Validation, Visualization, Project administration, Writing – original draft, Writing – review and editing.

Funding

This work received no specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The author is the primary developer of Burrete and may benefit reputationally from its adoption. The software is distributed under the MIT License. The author declares no other competing interests.

Data availability

No new research dataset was created for this article. Source code, documentation, sample files, release artifacts, and the manuscript source are available in the public GitHub repository [19].

Declaration of generative AI use

Generative AI tools were used to assist with drafting and editing this manuscript and with software-development workflows in the Burrete project. The author checked claims against source-controlled code and documentation, revised the generated material, and takes full responsibility for the final content. No generative AI was used to create or alter the scientific screenshots or the deterministic architecture diagram.

References

- [1] E. F. Pettersen, T. D. Goddard, C. C. Huang, E. C. Meng, G. S. Couch, T. I. Croll, J. H. Morris, T. E. Ferrin, UCSF ChimeraX: Structure visualization for researchers, educators, and developers, *Protein Science* 30 (1) (2021) 70–82. doi:10.1002/pro.3943.
- [2] P. Emsley, K. Cowtan, Coot: model-building tools for molecular graphics, *Acta Crystallographica Section D Biological Crystallography* 60 (12) (2004) 2126–2132. doi:10.1107/S0907444904019158.
- [3] W. Humphrey, A. Dalke, K. Schulten, VMD: visual molecular dynamics, *Journal of Molecular Graphics* 14 (1) (1996) 33–38. doi:10.1016/0263-7855(96)00018-5.
- [4] Schrodinger, LLC, The PyMOL molecular graphics system, version 3.0, citation format from the PyMOL project; accessed 2026-07-10 (2026). URL <https://www.pymol.org>
- [5] M. D. Hanwell, D. E. Curtis, D. C. Lonie, T. Vandermeersch, E. Zurek, G. R. Hutchison, Avogadro: an advanced semantic chemical editor, visualization, and analysis platform, *Journal of Cheminformatics* 4 (2012) 17. doi:10.1186/1758-2946-4-17.
- [6] N. M. O’Boyle, M. Banck, C. A. James, C. Morley, T. Vandermeersch, G. R. Hutchison, Open babel: An open chemical toolbox, *Journal of Cheminformatics* 3 (2011) 33. doi:10.1186/1758-2946-3-33.

- [7] G. Landrum, R. contributors, RDKit: Open-source cheminformatics, release 2025.03.4 (2025). doi:10.5281/zenodo.15773589.
- [8] D. Sehnal, S. Bittrich, M. Deshpande, R. Svobodova, K. Berka, V. Bazgier, S. Velankar, S. K. Burley, J. Koca, A. S. Rose, Mol* Viewer: modern web app for 3d visualization and analysis of large biomolecular structures, *Nucleic Acids Research* 49 (W1) (2021) W431–W437. doi:10.1093/nar/gkab314.
- [9] A. S. Rose, A. R. Bradley, Y. Valasatava, J. M. Duarte, A. Prlic, P. W. Rose, NGL viewer: web-based molecular graphics for large complexes, *Bioinformatics* 34 (21) (2018) 3755–3758. doi:10.1093/bioinformatics/bty419.
- [10] Jmol development team, Jmol: an open-source java viewer for chemical structures in 3d, accessed 2026-07-10 (2026). URL <https://jmol.sourceforge.net/>
- [11] N. Rego, D. Koes, 3dmol.js: molecular visualization with WebGL, *Bioinformatics* 31 (8) (2015) 1322–1324. doi:10.1093/bioinformatics/btu829.
- [12] Apple Inc., Quick Look, apple Developer Documentation; accessed 2026-07-10 (2026). URL <https://developer.apple.com/documentation/quicklook>
- [13] Tauri contributors, What is Tauri?, tauri documentation; accessed 2026-07-10 (2026). URL <https://v2.tauri.app/start/>
- [14] A. Midlik, S. Bittrich, J. R. Fleming, S. Nair, S. Velankar, S. K. Burley, J. Y. Young, B. Vallat, D. Sehnal, MolViewSpec: a Mol* extension for describing and sharing molecular visualizations, *Nucleic Acids Research* 53 (W1) (2025) W408–W414. doi:10.1093/nar/gkaf370.
- [15] B. Karulin, M. Kozhevnikov, Ketcher: web-based chemical structure editor, *Journal of Cheminformatics* 3 (S1) (2011) P3. doi:10.1186/1758-2946-3-S1-P3.
- [16] A. S. Goodfellow, xyzrender: Publication-quality molecular graphics (2026). doi:10.5281/zenodo.20478959.
- [17] K. Momma, F. Izumi, VESTA 3 for three-dimensional visualization of crystal, volumetric and morphology data, *Journal of Applied Crystallography* 44 (6) (2011) 1272–1276. doi:10.1107/S0021889811038970.

- [18] P. Eastman, J. Swails, J. D. Chodera, R. T. McGibbon, Y. Zhao, K. A. Beauchamp, L.-P. Wang, A. C. Simmonett, M. P. Harrigan, C. D. Stern, R. P. Wiewiora, B. R. Brooks, V. S. Pande, OpenMM 7: Rapid development of high performance algorithms for molecular dynamics, *PLOS Computational Biology* 13 (7) (2017) e1005659. doi:10.1371/journal.pcbi.1005659.
- [19] S. A. Nikolenko, Burrete: local-first molecular file inspection for macos, quick look, and agent workflows, version 1.0.22, computer software snapshot, application version 1.0.22, MIT License; accessed 2026-07-10 (2026).
URL <https://github.com/SergeiNikolenko/Burrete/tree/softwarex-v1.0.22>

Current executable software version

Version 1.0.22 is available from the GitHub release page: <https://github.com/SergeiNikolenko/Burrete/releases/tag/v1.0.22>. The release provides macOS DMG and ZIP artifacts; the iPhone target is distributed as source code.